

The objective of this lab is to:

Understand the role of Static members, use static data members and static member functions within a class, understanding their scope and lifetime relative to non-static members.

Instructions!

1. This is an individual lab, you are strictly NOT allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can ONLY discuss with your TAs or with me.
2. Strictly follow good coding conventions (commenting, meaningful variable and functions names, properly indented and modular code.

Question 1: NumberAnalyzer Class

Implement a **NumberAnalyzer** class that performs **logical analysis** on integers.

Class Header: NumberAnalyzer

```
class NumberAnalyzer {
private:
    int number;           // Stores the integer to analyze
    static int totalNumbers; // Static: counts total NumberAnalyzer objects

public:
    // Constructor: Initializes the number and increments object counter
    NumberAnalyzer(int n);

    // Destructor: Decrements the object counter
    ~NumberAnalyzer();

    // Logical/Analysis methods
    bool isPrime() const;           // Returns true if number is prime
    bool isEven() const;           // Returns true if number is even
    bool isArmstrong() const;       // Returns true if number is Armstrong
    bool isPerfect() const;         // Returns true if number is a perfect number
    bool isPalindrome() const;      // Returns true if number reads the same
    backward
    int digitSum() const;           // Returns sum of digits
    int reverseNumber() const;      // Returns the number reversed
}
```

```

    void displayProperties() const; // Displays all analysis results

    // Static method
    static int getTotalNumbers(); // Returns total NumberAnalyzer objects
};

```

Write a **main()** function that:

1. Creates at least **three NumberAnalyzer objects** with different integers.
 2. Calls the **displayProperties()** method for all.
 3. Displays the **total number of NumberAnalyzer objects** at the end.
-

Question 2: PasswordManager Class

Implement a **PasswordManager** class that manages passwords and evaluates their **strength** using logical conditions.

Class Header: PasswordManager

```

class PasswordManager {
private:
    string password;           // Stores the password string
    static int totalPasswords; // Static: counts all PasswordManager objects
    const int id;              // Constant unique ID for each object
    static int nextID;         // Static: generates unique IDs

public:
    // Constructor: Initializes password and assigns unique ID
    PasswordManager(string pass);

    // Destructor: Decrements totalPasswords counter
    ~PasswordManager();

    // Logical/Analysis methods
    bool validateLength() const; // Returns true if password length >=
8
    bool hasSpecialChar() const; // Returns true if password contains
non-alphanumeric character

```

```

    bool hasUpperLowerMix() const;           // Returns true if password has both
uppercase and lowercase letters
    bool hasNumbers() const;                 // Returns true if password contains
numbers
    int strengthScore() const;               // Returns strength score (0-10) based
on rules
    string encrypt() const;                  // Returns encrypted password
    bool match(string input) const;          // Returns true if input matches the
password
    void updatePassword(string newPass);     // Updates the password to a new value
    void displayInfo() const;                // Displays password info and strength

    // Static method
    static int getTotalPasswords();          // Returns total number of
PasswordManager objects
};

```

Write a **main()** function that:

1. Creates at least **three PasswordManager objects** with different passwords.
2. Calls **displayinfo()** for each object to Display password strength
3. Updates one password and demonstrates the change.
4. Displays the **total number of PasswordManager objects** at the end.